lab7

April 16, 2026

By: Rafay Siddiqui
Student No: 24K-0009
Section: BAI-4A

1 problem 1

A small retail store needs to assign three employees—Alice, Bob, and Charlie—to three distinct shifts: Morning (0), Afternoon (1), and Evening (2). Each employee must be assigned exactly one shift, and no two employees can work the same shift. Additionally, Alice is unavailable to work the Morning shift. Write a Python script using Google OR-Tools to model this Constraint Satisfaction Problem. Define the variables, apply the domains and constraints, and print the final consistent and complete assignment for all three employees.

1.1 Variables (V)

employees: $V = \{A, B, C\}$

1.2 Domains (D)

shifts for each employee (0=Morning, 1=Afternoon, 2=Evening): $D = \{0, 1, 2\}$

1.3 Constraints (C)

rules: - **Unique Shifts:** $A \neq B \neq C$ - **Alice's Restriction:** $A \neq 0$

[6]:
```python
from ortools.sat.python import cp_model

def solve_shift_assignment():
    model = cp_model.CpModel()

    alice = model.NewIntVar(0, 2, 'Alice')
    bob = model.NewIntVar(0, 2, 'Bob')
    charlie = model.NewIntVar(0, 2, 'Charlie')


    model.AddAllDifferent([alice, bob, charlie])

    model.Add(alice != 0)
```

```
    solver = cp_model.CpSolver()
    status = solver.Solve(model)

    if status == cp_model.OPTIMAL or status == cp_model.FEASIBLE:
        shift_names = {0: "Morning", 1: "Afternoon", 2: "Evening"}

        print(f"Alice's Shift:   {shift_names[solver.Value(alice)]}")
        print(f"Bob's Shift:     {shift_names[solver.Value(bob)]}")
        print(f"Charlie's Shift: {shift_names[solver.Value(charlie)]}")
    else:
        print("No consistent assignment found.")


solve_shift_assignment()
```

```
Alice's Shift:   Afternoon
Bob's Shift:     Evening
Charlie's Shift: Morning
```

2 problem 2

Cryptarithmetic is a classic logic puzzle where letters are replaced by numbers to form a valid mathematical equation. Write a Python script using Google OR-Tools to solve the equation: SEND + MORE = MONEY. You must define variables for the unique letters (S, E, N, D, M, O, R, Y) with a domain of 0 to 9. Apply constraints to ensure that every letter represents a strictly unique digit. Add constraints to prevent the leading letters ('S' and 'M') from being 0. Finally, create the linear constraint that mathematically represents the addition of the two words equaling the third. Print the integer value of each letter once the complete assignment is found.

2.1 1. Variables (V)

- Each unique letter represents a variable to be assigned a digit:
- $V = \{S, E, N, D, M, O, R, Y\}$

2.2 2. Domains (D)

- Each letter represents a single digit from 0 to 9:
- $D = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$

2.3 3. Constraints (C)

- The rules are defined by uniqueness, leading digits, and the arithmetic sum.
- **A. Uniqueness (AllDifferent):**
  - Every letter must be assigned a unique value:
  - All Different(S, E, N, D, M, O, R, Y)
- **B. Leading Digits:**

  - The first letter of a word cannot be zero:
  - $S \neq 0, \quad M \neq 0$

- **C. The Cryptarithmetic Equation:**
  - We expand the words into their base-10 positional values:
  - $(1000S + 100E + 10N + D) + (1000M + 100O + 10R + E) = 10000M + 1000O + 100N + 10E + Y$

[7]:
```
from ortools.sat.python import cp_model

def solve_cryptarithmetic():
    model = cp_model.CpModel()

    # 1. Define Variables
    # Unique letters: S, E, N, D, M, O, R, Y
    s = model.NewIntVar(0, 9, 'S')
    e = model.NewIntVar(0, 9, 'E')
    n = model.NewIntVar(0, 9, 'N')
    d = model.NewIntVar(0, 9, 'D')
    m = model.NewIntVar(0, 9, 'M')
    o = model.NewIntVar(0, 9, 'O')
    r = model.NewIntVar(0, 9, 'R')
    y = model.NewIntVar(0, 9, 'Y')

    letters = [s, e, n, d, m, o, r, y]

    # 2. Apply Constraints

    # Every letter must represent a unique digit
    model.AddAllDifferent(letters)

    # Leading letters cannot be zero
    model.Add(s != 0)
    model.Add(m != 0)

    # The equation: SEND + MORE = MONEY
    # 1000*S + 100*E + 10*N + D + 1000*M + 100*O + 10*R + E = 10000*M + 1000*O␣
↪+ 100*N + 10*E + Y
    model.Add(
        (1000 * s + 100 * e + 10 * n + d) +
        (1000 * m + 100 * o + 10 * r + e) ==
        (10000 * m + 1000 * o + 100 * n + 10 * e + y)
    )

    # 3. Solve
    solver = cp_model.CpSolver()
    status = solver.Solve(model)
```

```
    # 4. Print Results
    if status == cp_model.OPTIMAL or status == cp_model.FEASIBLE:
        print("Solution Found:")
        for letter in letters:
            print(f"{letter.Name()}: {solver.Value(letter)}")

        # Verify the sum
        send = 1000*solver.Value(s) + 100*solver.Value(e) + 10*solver.Value(n)␣
↪+ solver.Value(d)
        more = 1000*solver.Value(m) + 100*solver.Value(o) + 10*solver.Value(r)␣
↪+ solver.Value(e)
        money = 10000*solver.Value(m) + 1000*solver.Value(o) + 100*solver.
↪Value(n) + 10*solver.Value(e) + solver.Value(y)
        print(f"\nVerification: {send} + {more} = {money}")
    else:
        print("No solution found.")


solve_cryptarithmetic()
```

```
Solution Found:
S: 9
E: 5
N: 6
D: 7
M: 1
O: 0
R: 8
Y: 2

Verification: 9567 + 1085 = 10652
```

3 problem 3

A technology factory manufactures three products: Laptops, Phones, and Tablets. Each product requires a specific number of microchips, screens, and batteries. A Laptop requires 2 chips, 1 screen, and 1 battery, yielding a profit of $500. A Phone requires 1 chip, 1 screen, and 1 battery, yielding a profit of $300. A Tablet requires 1 chip, 1 screen, and 2 batteries, yielding a profit of $400. The factory has a limited daily inventory of exactly 100 microchips, 80 screens, and 120 batteries. Furthermore, company policy dictates that at least 10 units of each product must be manufactured daily to meet vendor contracts. Write an OR-Tools script to formulate this Integer Linear Programming (ILP) problem. Apply the inventory and minimum production constraints, and define the objective function to maximize total daily profit. Output the maximum profit alongside the optimal quantity of Laptops, Phones, and Tablets to produce.

3.1 1. Variables (V)

- L = Number of Laptops
- P = Number of Phones
- T = Number of Tablets

3.2 2. Domains (D)

Based on policy ($min \geq 10$) and inventory limits:

$$L, P, T \in \mathbb{Z}, \quad L, P, T \geq 10$$

3.3 3. Constraints (C)

The resource limits and production requirements: - **Chips:** $2L + 1P + 1T \leq 100$ - **Screens:** $1L + 1P + 1T \leq 80$ - **Batteries:** $1L + 1P + 2T \leq 120$

3.4 4. Objective Function

Maximize the total profit (Z):

$$\text{Maximize } Z = 500L + 300P + 400T$$

[8]:
```python
from ortools.sat.python import cp_model

def solve_factory_optimization():
    model = cp_model.CpModel()

    # Var (Production Quantities)
    # Min of 10 for each product as per policy
    laptops = model.NewIntVar(10, 100, 'Laptops')
    phones = model.NewIntVar(10, 100, 'Phones')
    tablets = model.NewIntVar(10, 100, 'Tablets')

    # Constr. (Resource Inventory)

    # Microchips: 2 per Laptop, 1 per Phone, 1 per Tablet (Max 100)
    model.Add(2 * laptops + 1 * phones + 1 * tablets <= 100)

    # Screens: 1 per Laptop, 1 per Phone, 1 per Tablet (Max 80)
    model.Add(1 * laptops + 1 * phones + 1 * tablets <= 80)

    # Batteries: 1 per Laptop, 1 per Phone, 2 per Tablet (Max 120)
    model.Add(1 * laptops + 1 * phones + 2 * tablets <= 120)

    # 3Obj Func (Maximize Profit)
    # Profit = 500*L + 300*P + 400*T
    model.Maximize(500 * laptops + 300 * phones + 400 * tablets)
```

```
    solver = cp_model.CpSolver()
    status = solver.Solve(model)


    if status == cp_model.OPTIMAL:
        print(f"Maximum Daily Profit: ${solver.ObjectiveValue()}")
        print(f"--- Production Plan ---")
        print(f"Laptops: {solver.Value(laptops)}")
        print(f"Phones:  {solver.Value(phones)}")
        print(f"Tablets: {solver.Value(tablets)}")
    else:
        print("No optimal solution found.")


solve_factory_optimization()
```

```
Maximum Daily Profit: $32000.0
--- Production Plan ---
Laptops: 23
Phones:  11
Tablets: 43
```